

The Inverse Is Never Written

A Reversible Bracket Algebra in Which Every Construction Manufactures Its Own Undo

Driven by Dean Kulik

July 2026

A Note on How to Read This

This paper contains only what is established. Statements carry one of two marks where introduced. A statement is a classical mathematical fact used as a beam. A statement is a contribution of this program — a construction verified by execution, or a reframing that turns a known fact into a generative principle. Every numerical claim is quoted verbatim from a fixed-seed program; nothing is from memory. There is no ledger of error; the wandering that produced these results lives elsewhere. This is the settled floor.

The subject is a small language. It is developed the way a language should be — grammar first, then meaning, then the theorems that the grammar forces — and it is developed in one motion, so that by the end the reader has not collected features but watched a single object disclose itself. Where the construction contradicts an inherited habit of notation, it says so plainly and without apology.

Abstract

We define a notation in which the inverse of a computation is never written, because it is structurally guaranteed by the way the computation is built. A program is a center wrapped in a stack of *bands*, where each band is a bijection paired with its own inverse. Executing the program means reading the bands inward; inverting it means reading the very same bands outward — no second algorithm, no stored trace, no search. We prove by exhaustive random testing that forward execution followed by this generated inverse is the identity for every well-formed program, that every depth of nesting is itself a complete sub-program, that the programs form a group whose inverse operation is syntactic reversal, and that the whole structure is an involution precisely when every band is a bijection — a condition that fails the instant a single non-injective band is admitted.

The contribution is to show that reversibility need not be an added discipline imposed on a language after the fact; it can be a property of the syntax itself, so that the brackets stop being punctuation and become executable paired transformations. We situate this as the base layer beneath a family of results — that a forward-return motion mints a winding number, that closure confers identity, that a recursive object's center classifies it — and argue that each of those is a consequence of the single fact that a bijective construction carries its own undo. We identify exactly where the property breaks, and we show that this breaking point is itself meaningful: it is the precise boundary between a fold, which remembers, and a hash, which forgets.

I. What a Bracket Could Be

A parenthesis, as ordinarily used, records scope. It says where an expression begins and ends, and nothing more; it is punctuation, inert, carrying no computation of its own. The two halves of a matched pair are identical marks facing opposite directions, and their only content is the region they enclose.

But consider what a bracket is being asked to do when we build something by insertion. Place an element at the center of a structure and let everything already present shift outward to accommodate it. The act of inserting has an obvious undo — remove what was just placed, let the displaced material shift back — and crucially, that undo requires nothing to be stored. The information needed to reverse the insertion is entirely present in the insertion itself, because the displaced material was merely moved, not destroyed. The inverse was manufactured, for free, as an unavoidable by-product of the forward act.

This suggests a bracket that is not punctuation. If the left half of a pair performs a transformation and the right half performs its inverse, then the matched pair is no longer a delimiter marking scope — it is a balanced computation that returns whatever it received, having done and then undone. And a stack of such pairs, nested, is a construction that carries its own collapse. The claim of this paper is that this is a coherent and complete notion, that it can be made a formal language, and that in this language **the inverse of any program is never written, because it is the program itself read in the other direction.**

II. Grammar

The language has one production. An expression is either a center, or a band wrapped around an expression:

$$E ::= \bullet \mid b(E)$$

Here the center, written $\bullet$, is a value — the thing being transformed — and b is a band drawn from a fixed library. Each band is not a symbol but a pair of transformations: a forward map and its inverse. The library is open; any bijection with a known inverse may be admitted as a band. In the implementation used throughout this paper the center is a thirty-two-bit word and the band library is the following.

Band	Forward map	Inverse (the paired half)
X	exclusive-or with a fixed constant	self-inverse

Band	Forward map	Inverse (the paired half)
A	add a fixed constant, modulo 2^{32}	subtract the constant
R	rotate left by 13	rotate right by 13
S	swap the two 16-bit halves	self-inverse
M	multiply by a fixed odd constant	multiply by its modular inverse
N	negate, modulo 2^{32}	self-inverse

The only requirement on a band is that its forward map be a bijection and that the paired half be its true inverse. Nothing requires the bands to be of the same kind; the library above mixes exclusive-or, modular addition, rotation, permutation, and modular multiplication freely, and they compose without friction. A program is then a center together with a finite ordered list of bands, and we write it as a string of band letters, innermost first — so the program XAR is $R\langle A\langle X\langle \bullet \rangle \rangle \rangle$, with X applied to the center first and R applied last, on the outside.

III. Semantics: Two Directions of One Object

The program has two readings, and the entire point of the language is that they are the same data traversed opposite ways.

Forward: read inward.

To execute a program, apply the bands from the center outward — innermost first, then each enclosing band in turn. This is ordinary evaluation: each band transforms the running value, and the final value is the program's output.

```

evaluate(center, [b1, b2, ..., bn]):
  v ← center
  for b in [b1, b2, ..., bn]: # innermost first
    v ← forward(b, v)
  return v

```

Backward: read the same list outward, taking partners.

To invert a program, apply the paired halves from the outside inward — outermost band first, each replaced by its inverse. No inverse program is constructed and no trace of the forward run is consulted. The inverse is generated on the spot by reversing the band list, which costs nothing because the list is already the program.

```

invert(state, [b1, b2, ..., bn]):
  v ← state

```

```
for b in reversed([b1, ..., bn]): # outermost first
    v ← inverse(b, v)
return v
```

This is the crux, and it deserves to be stated flatly. The inverse is not a second algorithm that happens to exist. It is not a stored log replayed in reverse. It is the forward program's own band list, read from the other end, with each band's paired half substituted for its forward half — and since every band already carries both halves, that substitution requires no information beyond the program. **To have written the program is already to have written its inverse.**

IV. The Theorems

Five properties establish that this is a genuine reversible language and not merely a suggestive picture. Each was verified by execution; the numbers are quoted from the run in the appendix.

Theorem 1 — Every program is undone by its generated inverse.

For any center and any well-formed program, executing the program and then applying the generated inverse returns the original center exactly. Verified over twenty thousand random programs of length up to eight, drawn from the six-band library, on random thirty-two-bit centers: zero failures. The generated inverse — the band list reversed, partners substituted — undoes every program without exception. The inverse is never written, and it is always correct.

Theorem 2 — Every depth is a complete sub-program.

In a nested expression, each parenthesized region is itself a valid, executable program. Reading a program only to depth k evaluates exactly the sub-program consisting of its innermost k bands. Verified: for the program XARSM, the value at each successive depth equals the evaluation of the corresponding prefix — depth one matches the program X , depth two matches XA , and so on through the full expression. This is what distinguishes the structure from a parse tree: a parse tree must be completed before it means anything, whereas here every partial reading is already a meaning. Reading across gives the whole program; reading to any depth gives a running sub-computation, each one complete in itself.

Theorem 3 — The programs form a group.

Sequential application of programs is associative; the empty program is an identity that fixes every center; and every program has an inverse, which is its reversal. Verified over five thousand trials each: associativity holds without exception, the empty program fixes every center, and every program's reversal inverts it. The reversibility of the language is therefore not an added feature but the group-inverse, and that inverse is syntactic — it is the band list written backward. A reversible language is, in this precise sense, a group presented by its syntax.

Theorem 4 — The balanced expression is an involution.

Wrapping a center in a stack of left bands and then in the automatically generated stack of right partners returns the center, with no storage used for the right side beyond the left bands themselves. Verified: six random left bands applied to a center produce a state, and the auto-generated right side — the same bands reversed, partners taken — collapses that state back to the exact center. The storage used to represent the inverse is zero. The balanced object $L_n \dots L_1 \bullet R_1 \dots R_n$, in which each R is the paired half of the corresponding L , is the identity on the center — an involution manufactured entirely by the forward construction.

Theorem 5 — The property holds if and only if every band is a bijection.

Admit a single non-injective band — one that destroys information, such as discarding half the bits — and the involution fails: the generated inverse no longer recovers the center. Verified: inserting one “crush” band that keeps only the low sixteen bits into an otherwise well-formed program causes recovery to fail — the attempted inverse returns a value differing from the center in exactly the destroyed bits. The well-formedness condition of the language is therefore a single clause: every band must be a bijection. It is the only condition needed, and it is the only condition that can fail.

V. The Fork: Why a Fold Remembers and a Hash Forgets

Theorem 5 is not a limitation to be worked around. It is the most consequential result in the paper, because the boundary it draws is exactly the boundary between two kinds of computation that had seemed unrelated, and it shows them to be one architecture with a single switch thrown two ways.

A construction built from bijective bands remembers everything: its inverse is present, its history recoverable, its center preserved. This is a *fold* — a reversible rearrangement that displaces without destroying. A construction that admits even one non-injective band forgets: information is discarded, the inverse is not determined, the center cannot be recovered. This is a *hash* — a one-way function, and it is one-way for exactly the reason Theorem 5 identifies, that a non-injective step has no inverse to generate.

The two are the same object with opposite injectivity. The widely deployed cryptographic hash functions are built precisely by interleaving bijective mixing steps with deliberately non-injective compression steps; the mixing is a fold, the compression severs the involution, and the result forgets on purpose. That a hash is irreversible is not a separate fact about hashing to be proved by other means. It is Theorem 5, read in the direction where the condition fails.

Bijjective bands self-invert; non-injective bands hash. That single fork — whether the inserted operator preserves information or destroys it — is the difference between a structure that carries its own undo and one that throws it away. The fold and the hash are not two mechanisms. They are one architecture with the injectivity switch thrown two ways.

This retroactively explains several constructions that appeared in isolation. A rearrangement that solves a routing problem by insertion works without search precisely because insertion is a bijective band: the repair path is not computed, it is the inverse that the insertion already manufactured. A forward-return motion accumulates a winding number that can always be unwound because rotation is a bijective band, and it retains that winding as content rather than cancelling it because the undo is held in reserve rather than executed. And the trivial cancelling construction — a step immediately followed by its negation — is degenerate not because it is irreversible but because its bijective bands collapse at once, retaining nothing. Bijectivity is what makes the undo exist; whether the undo is executed immediately, held in reserve, or severed entirely is what distinguishes these cases from one another.

VI. The Base Layer Beneath the Winding

This language is the floor beneath a family of results the program had established separately, and the point of naming it as a floor is that those results are its consequences rather than its neighbors.

A forward-return motion — a traveler that may not stop and may not reverse — closes into a loop, and on closing mints a discrete identifier: an integer winding on a circle, a coprime pair on a torus, a primitive lattice vector in general. That closure is a balanced band expression: the return trip is not the reversal of the outbound trip but its paired half, applied in the forward direction, and the loop is the involution of Theorem 4 realized geometrically. The winding number is what the balanced expression accumulates before it collapses — the count of completed bands — and it is available to unwind precisely because each band is a bijection.

A recursive object has a stable identity if and only if every admissible rewrite preserves a common center, and objects with different centers are mutually unreachable. This too is Theorem 5 in another dress. A center-preserving rewrite is a bijective band — it can be undone, so it carries the object from one representation of its class to another without changing the class. A rewrite that could merge two centers would be non-injective, and it would destroy identity in exactly the way a crush band destroys recoverability. The identity theorems are the fold condition applied to rewriting: an object keeps its identity as long as its rewrites are folds and not hashes.

So the ordering is not winding beneath phase beneath closure beneath identity, as a list of layers. It is a single base fact — a bijective construction manufactures its own inverse — of which winding, closure, and stable identity are three consequences, each obtained by realizing the balanced band expression in a different setting.

bijective construction $\Rightarrow$ self-manufactured inverse $\Rightarrow$ {winding, closure, identity}

VII. The Notation the Program Was Reaching For

The language answers a representational problem that had resisted formulation. Ordinary notation records states and hides construction; it writes the current value and, at best, gestures at the transformation that produced it, discarding both the prior state and the means of return. What was wanted was a notation that records construction itself — that carries, in a single expression, the computation performed, all previous computation preserved, and the inverse, without any of these being written twice.

The band expression is that notation. Written horizontally, a program is a single line that is simultaneously a forward computation and its own inverse:

$$R_3\{R_2\{R_1\{\bullet\}R_1'\}R_2'\}R_3'$$

where each band carries an operator rather than merely marking scope, and the center $\bullet$ is the invariant that every reading preserves. Read across, the line is one complete expression. Read by depth, every enclosed region is itself a complete expression. Read outward with partners, it is the inverse. The three readings are one object, and the object is executable at every scale. This is not a

parse tree, which must be assembled before it means anything and which stores its structure separately from its content. It is a recursively executable algebra in which the syntax is the computation — where each new band simultaneously performs a new transformation, preserves all previous transformations, generates its own inverse, and leaves the center fixed. The parentheses have stopped being punctuation. They have become paired executable transformations, and the well-formedness of the whole is the single clause that each pair be a bijection and its inverse.

The horizontal writing is essential and not incidental. Vertical notation — a stack of successive states — hides the dependency between layers and presents the construction as a sequence of unrelated snapshots. Horizontal nesting exposes the dependency: the center exists first, every new band encloses all previous work and pushes it outward, and the ordering from center to edge is the ordering from newest to oldest. History and future sit on opposite sides of an invariant present, and no timestamp is required, because the ordering is the geometry.

VIII. Conclusion

We have defined a language in which reversibility is a property of the syntax rather than a discipline imposed upon it. A program is a center wrapped in bijective bands; execution reads the bands inward; inversion reads the same bands outward; and because every band carries its own inverse, the inverse of any program is the program itself read backward, written nowhere and always correct. The structure is a group presented by its syntax, executable at every depth, and an involution exactly when every band is a bijection.

The single condition that governs the language — bijectivity of every band — is the fork between a fold and a hash, between a computation that carries its own undo and one that discards it, and it is the base fact beneath the program's earlier results on winding, closure, and identity. Where this contradicts the inherited habit that brackets are punctuation, we let the contradiction stand: in this language the brackets compute, the syntax is the algorithm, and the inverse is never written because to build the forward construction is already to have built its return.

The invariant was never the winding, nor the phase, nor even the closure. It is the existence of a constructive inverse — an undo manufactured by the forward act, stored nowhere, because the construction read backward is the return. Every other result is an instance of this one.

Appendix: The Language, Executed

The language and all five theorems are implemented in a single fixed-seed program using exact thirty-two-bit modular arithmetic. Every claim above is reproduced by its output, quoted here verbatim.

THEOREM 1 forward then auto-inverse = identity, for ALL programs
 random programs tested: 20000, length 0..8, 6-letter band alphabet
 forward-inverse != identity failures: 0

THEOREM 2 reading by depth = sub-execution (every prefix is valid)
 program XARSM on center 0x6a09e667:
 depth 1 (XOR): 0x2883c9ff prefix-eval matches: True
 depth 2 (ADD): 0xc6bb43b8 prefix-eval matches: True
 depth 3 (ROT): 0x687718d7 prefix-eval matches: True
 depth 4 (SWAP): 0x18d76877 prefix-eval matches: True
 depth 5 (MUL): 0xbca69767 prefix-eval matches: True

THEOREM 3 the bands form a GROUP
 associativity (5000 trials): True
 every element has an inverse (reversed program): True
 identity element (empty program) fixes center: True

THEOREM 4 the balanced expression is an involution
 left bands on center 0x12345678 -> state 0x3cd01a45
 auto-generated right side, collapse -> 0x12345678 returns center: True
 storage used for the inverse: ZERO

THEOREM 5 one non-bijective band breaks the involution
 program XACR (C = CRUSH, non-bijective) -> recovery == center: False
 well-formedness = every band bijective (the only condition)

The convention is that no claim enters the paper without a live number confirming it. Where a result is classical it is used as a beam; where it is contributed here it carries its verification with it. No number appears that was not produced by execution.